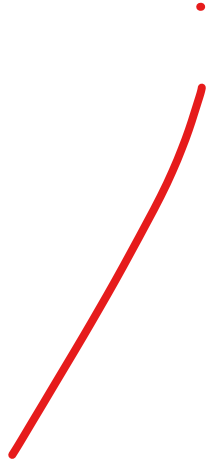
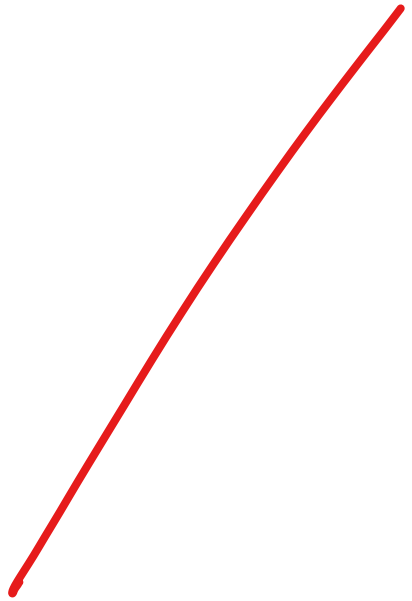
Priority Queue and Heap



PRIORITY QUEUE



Priority Queue

- Normal Queue: (Consider the values as age)

- Priority Queue:

Priority Queue

- Priority Queue: (Priority given to the seniors)

Priority Queue

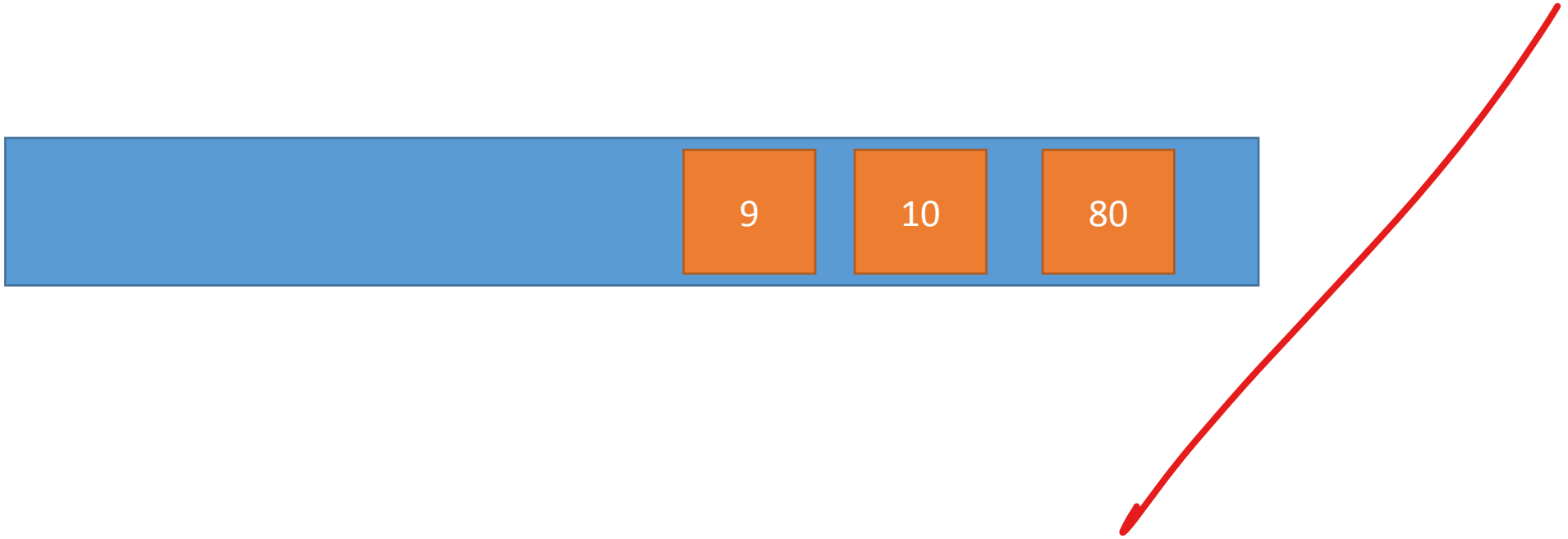
- Priority Queue: (Priority given to the seniors)

Priority Queue

- Priority Queue: (Priority given to the seniors)

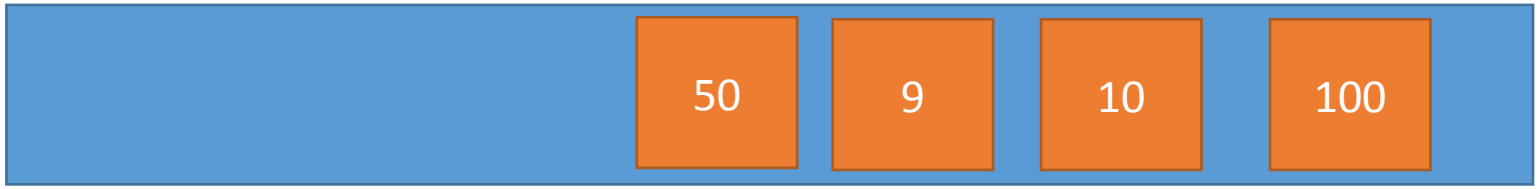
Priority Queue

- Priority Queue: (Priority given to the seniors)



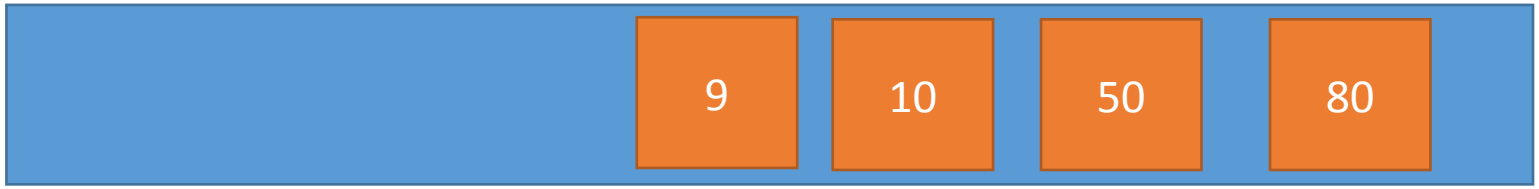
Priority Queue

- Priority Queue: (Priority given to the seniors)



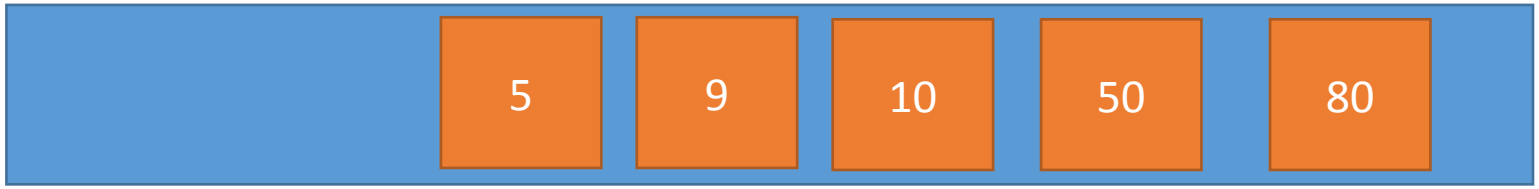
Priority Queue

- Priority Queue: (Priority given to the seniors)



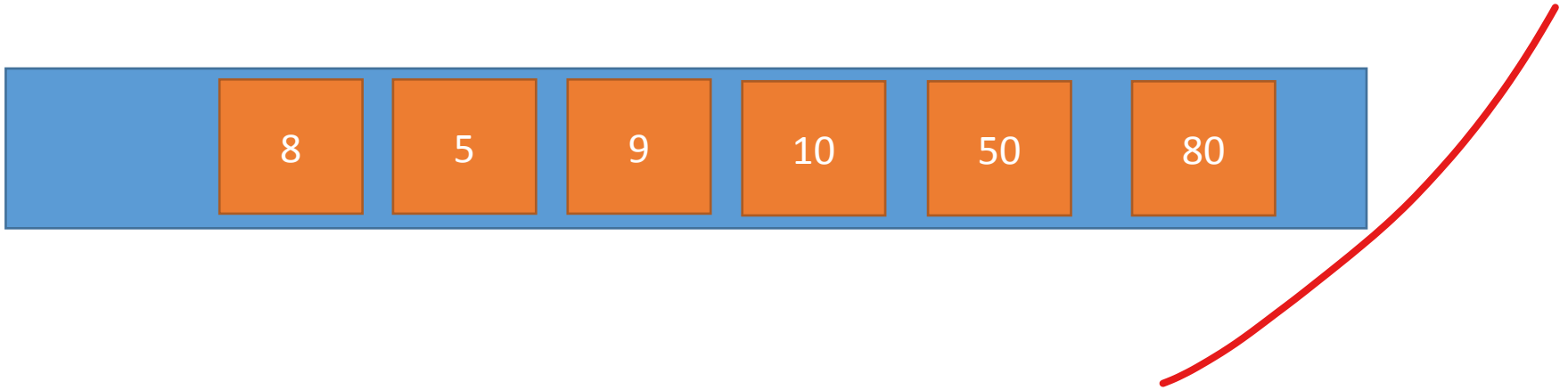
Priority Queue

- Priority Queue: (Priority given to the seniors)



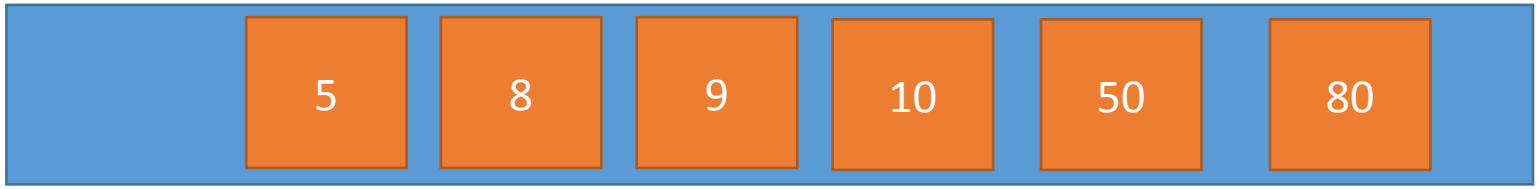
Priority Queue

- Priority Queue: (Priority given to the seniors)



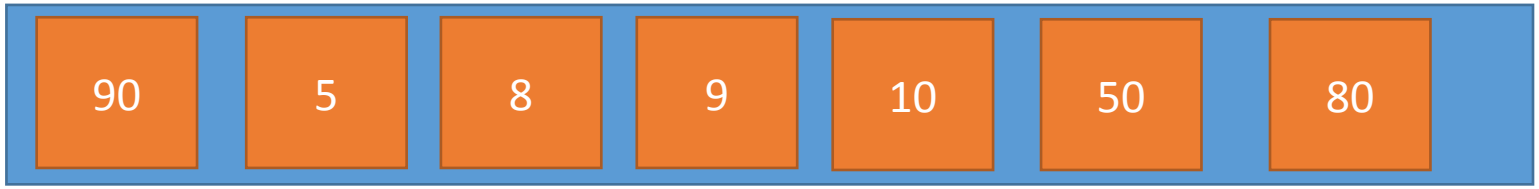
Priority Queue

- Priority Queue: (Priority given to the seniors)



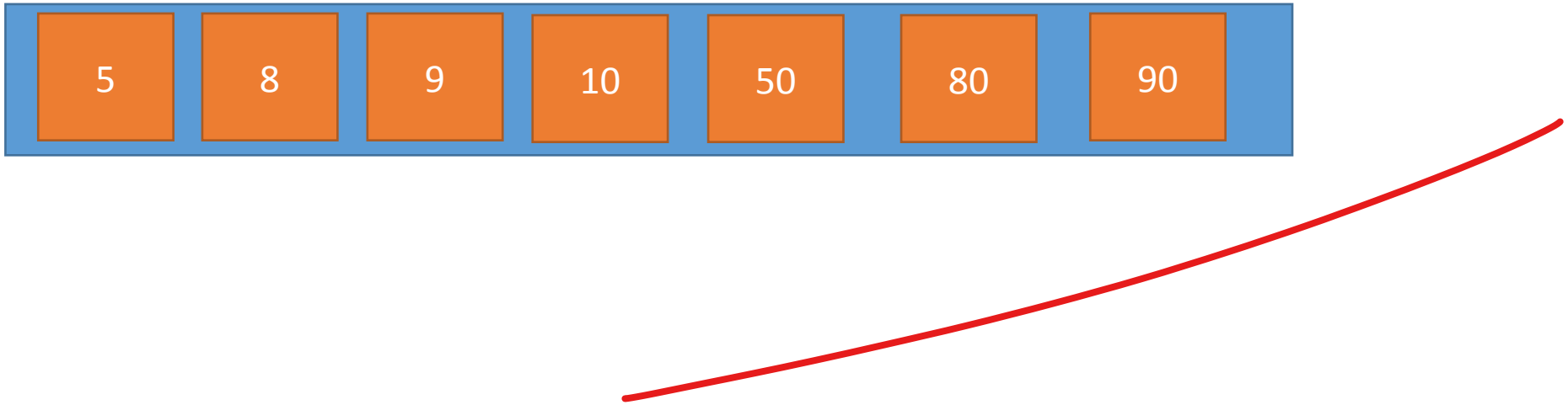
Priority Queue

- Priority Queue: (Priority given to the seniors)



Priority Queue

- Priority Queue: (Priority given to the seniors)



Applications of Priority Queue

- Line-up of Incoming Planes at Airport
  - VIP Planes
  - Commercial Planes
  - Emergency Planes
- Operating Systems Priority Queues
  - Alarm clock app
  - Messenger app

BINARY HEAP

Binary Heaps

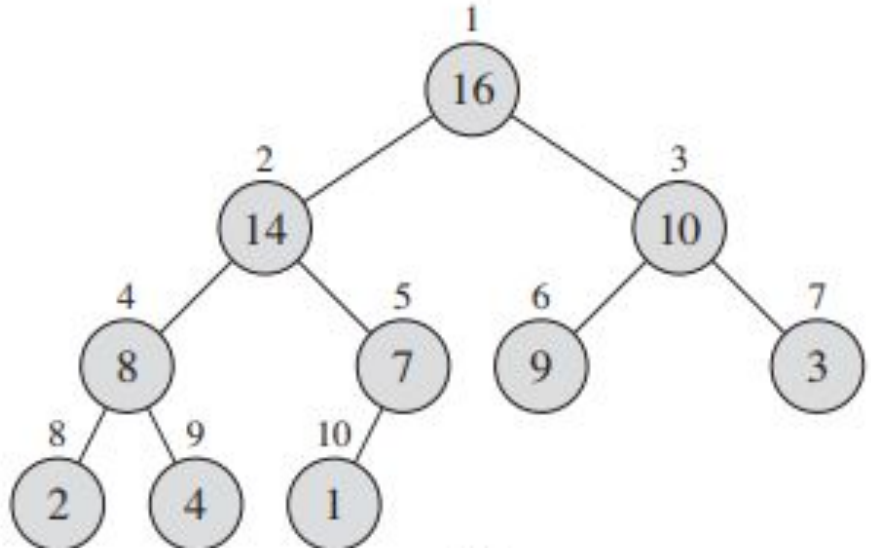
- The (binary) heap data structure is an **array object** that can be viewed as a **complete binary tree.**
  - Each node of the tree corresponds to an element of the array that stores the value in the node.
  - The tree is completely filled on all levels except possibly the lowest, where it is filled from the left up to a point.

- Despite the fact that it is in fact an **array.**
- We will **imagine** the entire data structure to be a **tree.**

Binary Heaps

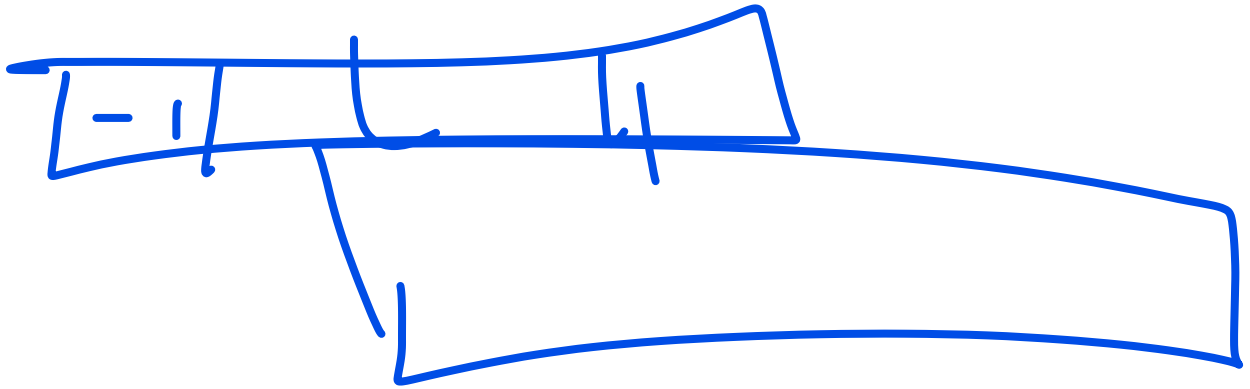
- How can we visualize this array as a complete binary tree?

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1



Binary Heaps

- Some **basic** variables and functions of a heap:
  - A.heap-size
  - A.length
  - PARENT(i)
  - LEFT(i)
  - RIGHT(i)
- Let us observe the array again



1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

Binary Heaps

- Question: Who is the parent of the 7th array element?

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

- Answer:
  - We can draw the tree and find out
  - We can use these functions and determine directly

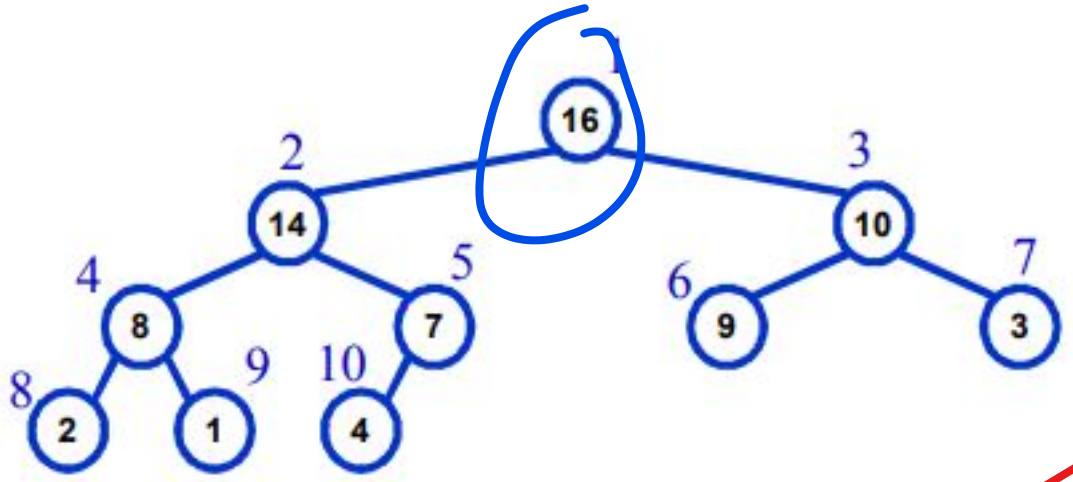
```
PARENT(i)
1  return ⌊i/2⌋
```

```
LEFT(i)
1  return 2i
```

```
RIGHT(i)
1  return 2i + 1
```

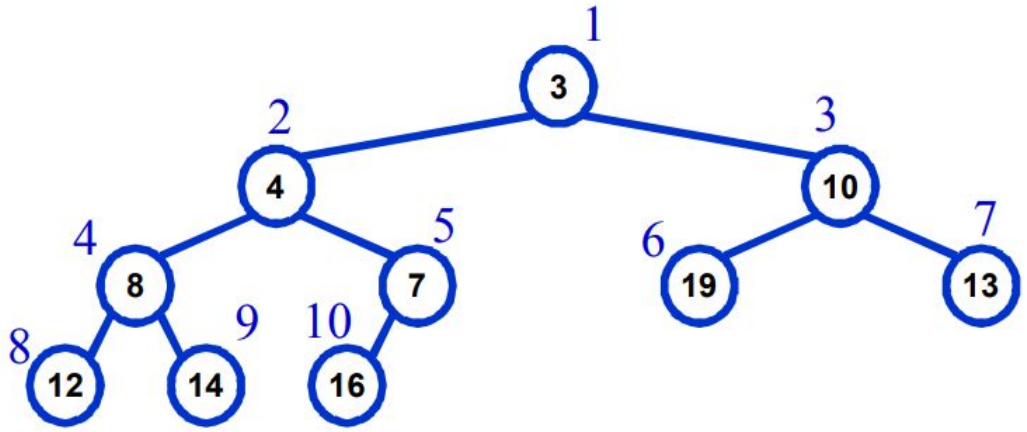
Max Heap and Min Heap

- Max-Heaps
  - Satisfies the heap property: **A[Parent(i)] >= A[i]** for all nodes i > 1
    - The value of a node is at most the value of its parent
  - The **largest element** in a max-heap is stored at the **root**
  - Where is the **smallest element** ?
    - Ans: At one of the **leaves** [leaf indices are n/2+1 to n]



Max Heap and Min Heap

- Min-Heaps
  - Satisfies the heap property: **A[Parent(i)] <= A[i]** for all nodes i > 1
    - The value of a node is at least the value of its parent
  - The **smallest element** in a max-heap is stored at the **root**
  - Where is the **largest** element ?
    - Ans: At one of the leaves [leaf indices are n/2+1 to n]



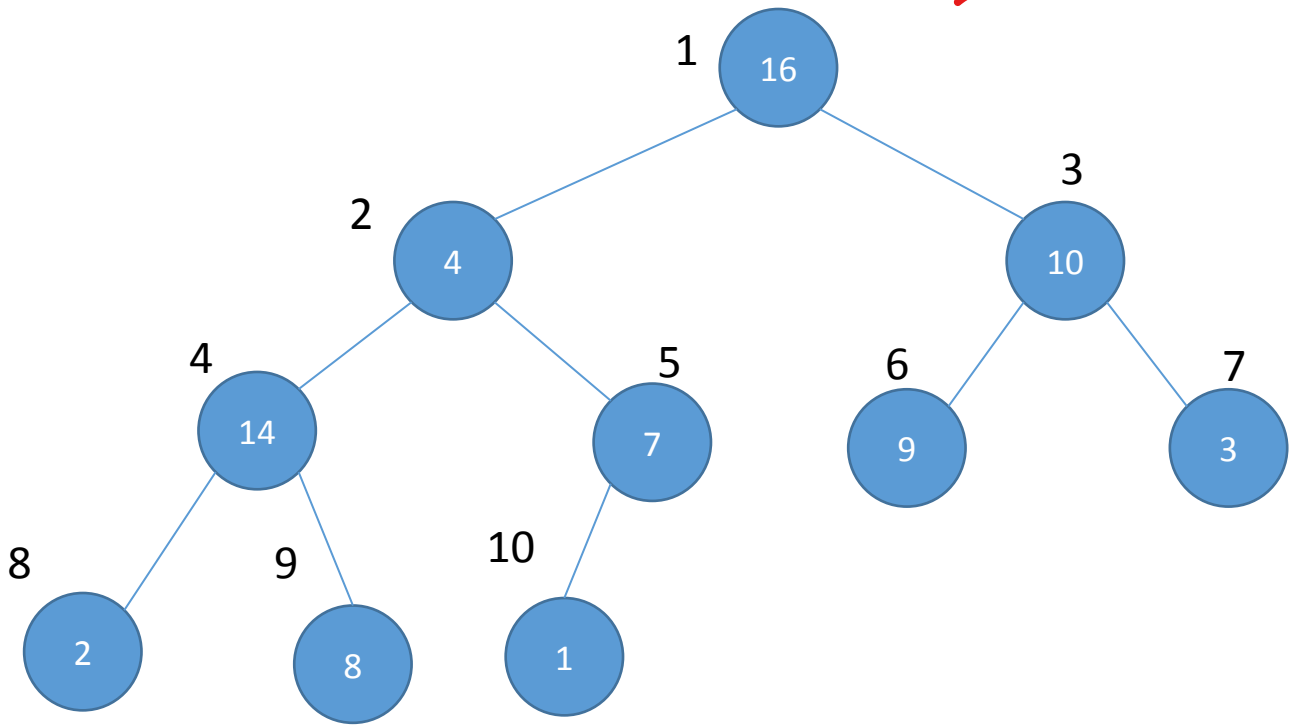
Important Functions of Max Heap

- **MAX-HEAPIFY –**
  - **O(lg n)** time
  - **Tries** to make the array into a heap
- **BUILD-MAX-HEAP**
  - **O(n)** time
  - Converts an array into a heap
- **HEAPSORT**
  - O **(n lg n)** time
  - **Another sorting algorithm**
  - Makes use of the concept of heap
- **Priority Queue functions**
  - **MAX-HEAP-INSERT**
  - **HEAP-EXTRACT-MAX**
  - **HEAP-INCREASE-KEY**
  - **HEAP-MAXIMUM**
  - run in O(lg n) time

MAX HEAPIFY

- **Tries** to make the array into a max heap
- (parent value>= child value)
- MaxHeapify(A,index)

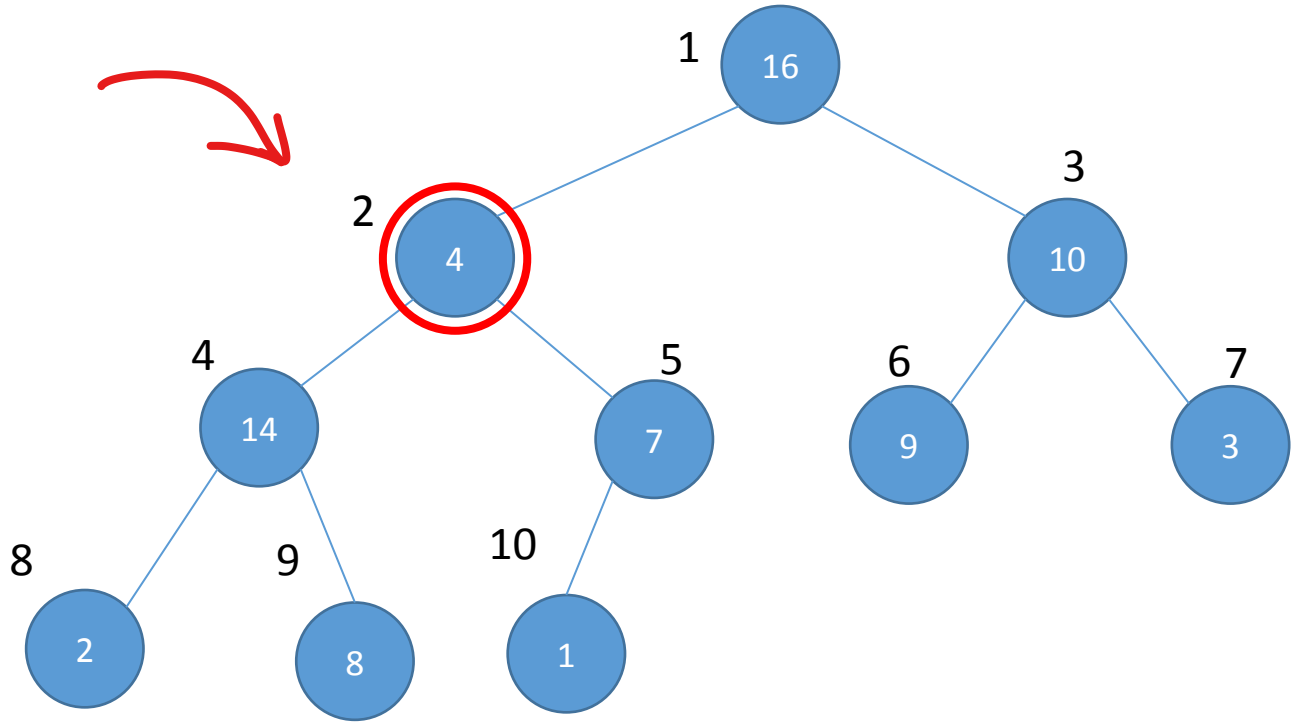
Created by BuildHeap
No Sort



1	2	3	4	5	6	7	8	9	10
16	4	10	14	7	9	3	2	8	1

MAX HEAPIFY

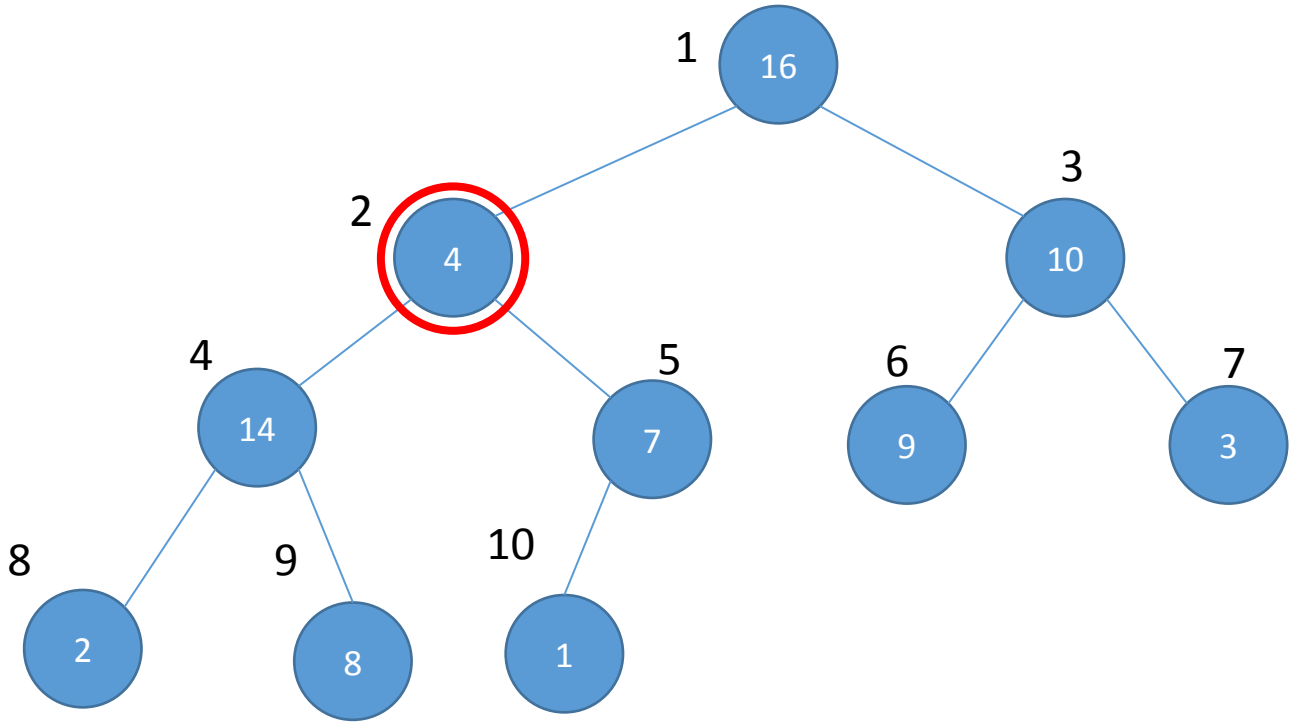
- **Tries** to make the array into a max heap
- (parent value>= child value)
- MaxHeapify(A,index)



1	2	3	4	5	6	7	8	9	10
16	4	10	14	7	9	3	2	8	1

MAX HEAPIFY

- **Tries** to make the array into a max heap
- (parent value>= child value)
- MaxHeapify(A,index)



1	2	3	4	5	6	7	8	9	10
16	4	10	14	7	9	3	2	8	1

How the function works:

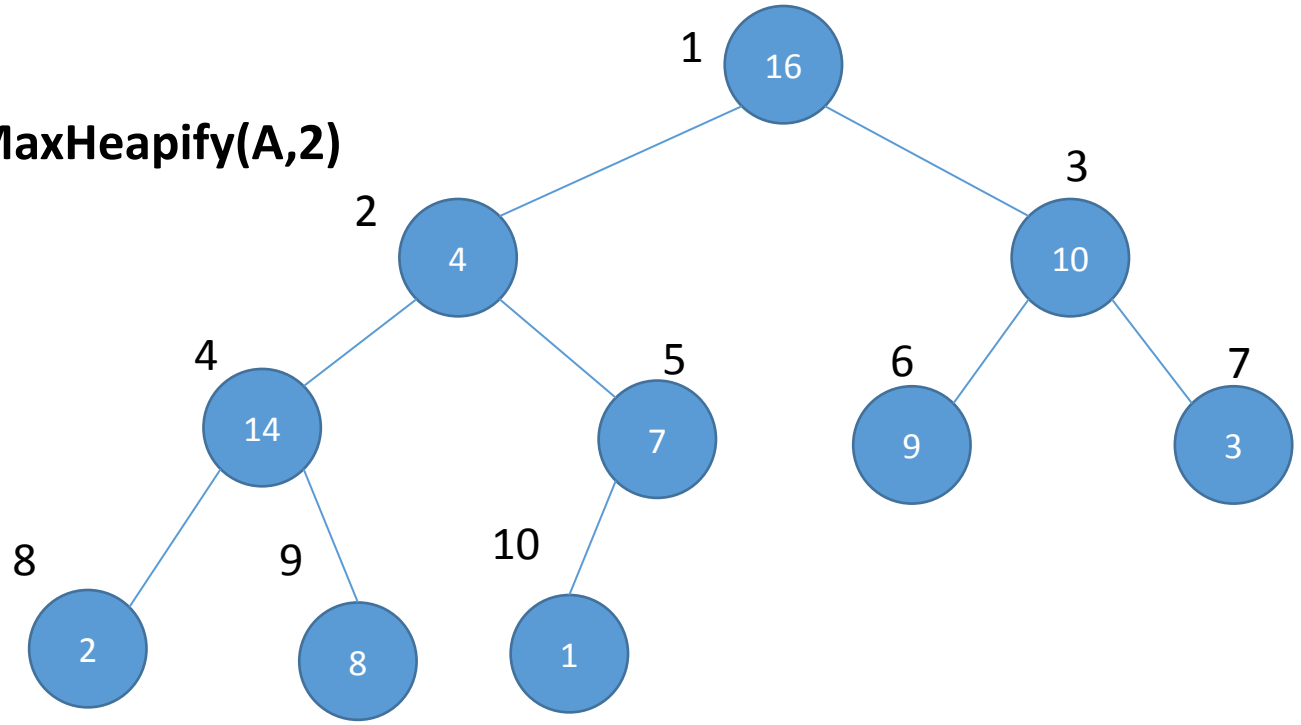
1. It is applied on an index.
2. It checks if the parent is greater than the child
3. If the parent is greater, return.
4. If the child is greater, then swap values with child
   a) Run the function on the child (recursively)

MAX HEAPIFY

- **Tries** to make the array into a max heap
- (parent value>= child value)
- MaxHeapify(A,index)

1	2	3	4	5	6	7	8	9	10
16	4	10	14	7	9	3	2	8	1

MaxHeapify(A,2)



How the function works:

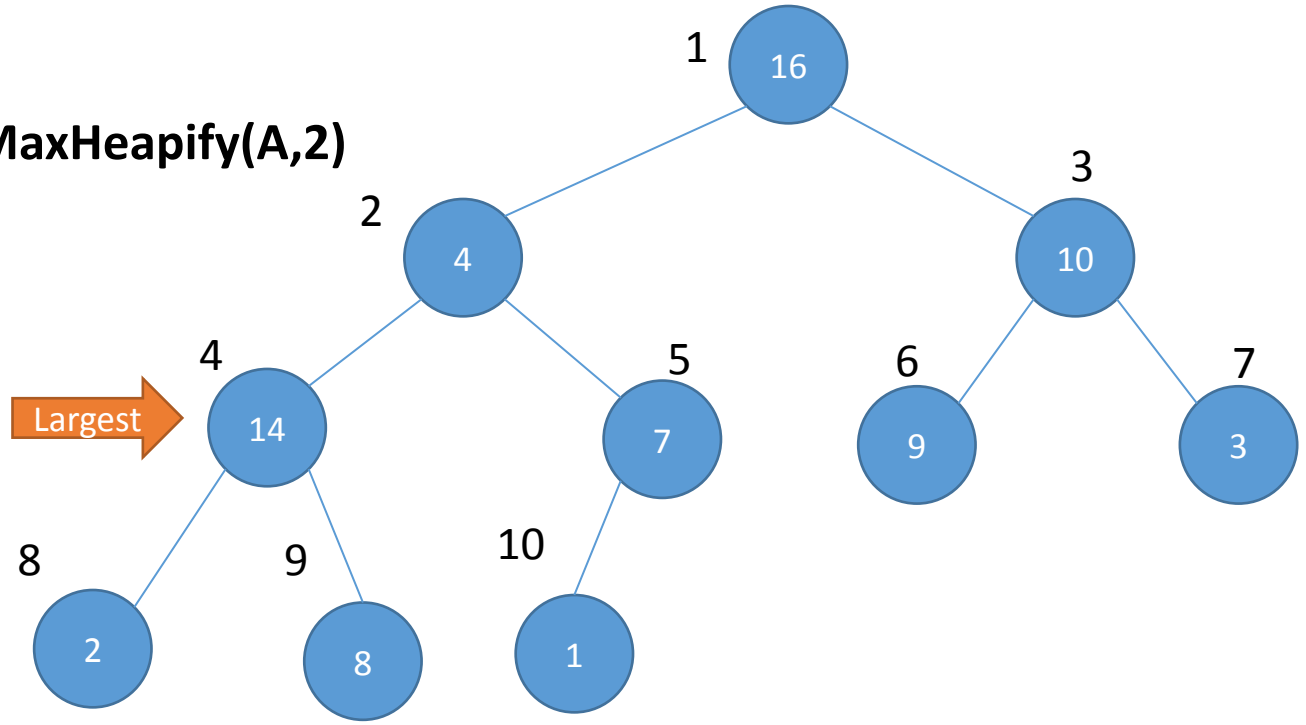
1. It is applied on an index.
2. It checks if the parent is greater than the child
3. If the parent is greater, return.
4. If the child is greater, then swap values with child
   a) Run the function on the child (recursively)

MAX HEAPIFY

- **Tries** to make the array into a max heap
- (parent value>= child value)
- MaxHeapify(A,index)

1	2	3	4	5	6	7	8	9	10
16	4	10	14	7	9	3	2	8	1

MaxHeapify(A,2)



How the function works:

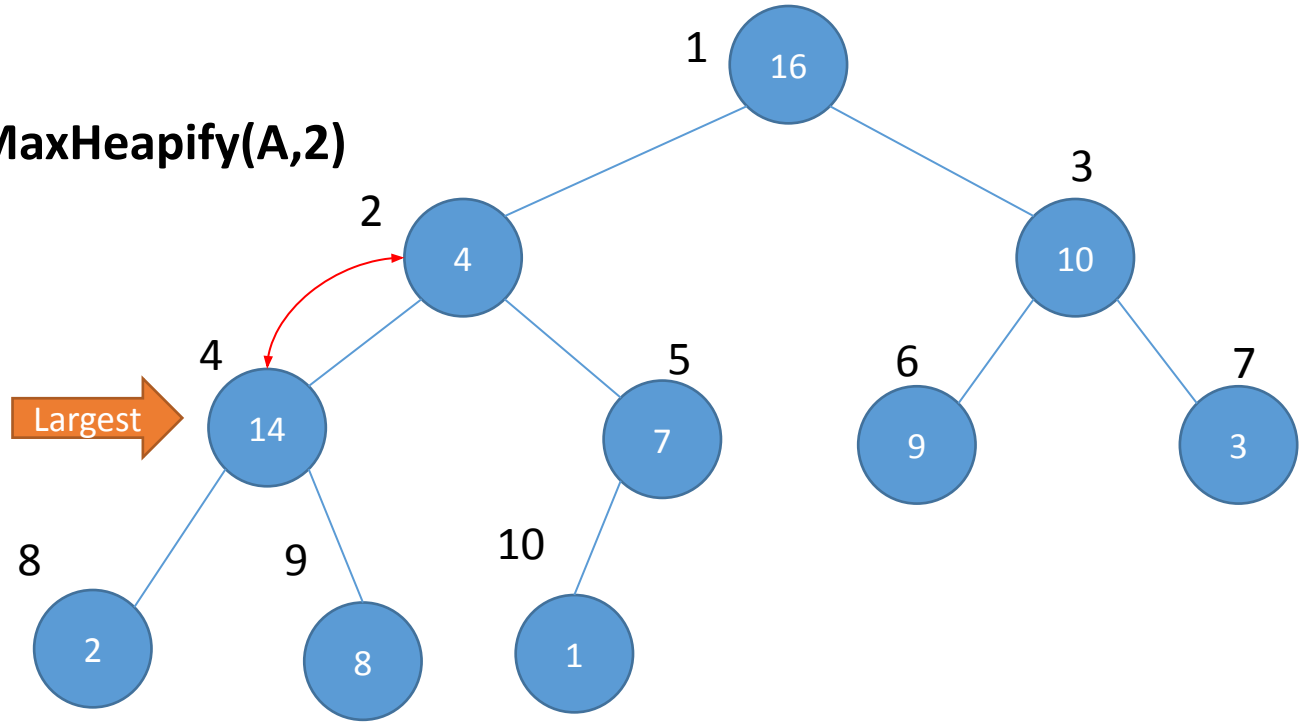
1. It is applied on an index.
2. It checks if the parent is greater than the child
3. If the parent is greater, return.
4. If the child is greater, then swap values with child
   a) Run the function on the child (recursively)

MAX HEAPIFY

- **Tries** to make the array into a max heap
- (parent value>= child value)
- MaxHeapify(A,index)

1	2	3	4	5	6	7	8	9	10
16	4	10	14	7	9	3	2	8	1

MaxHeapify(A,2)



How the function works:

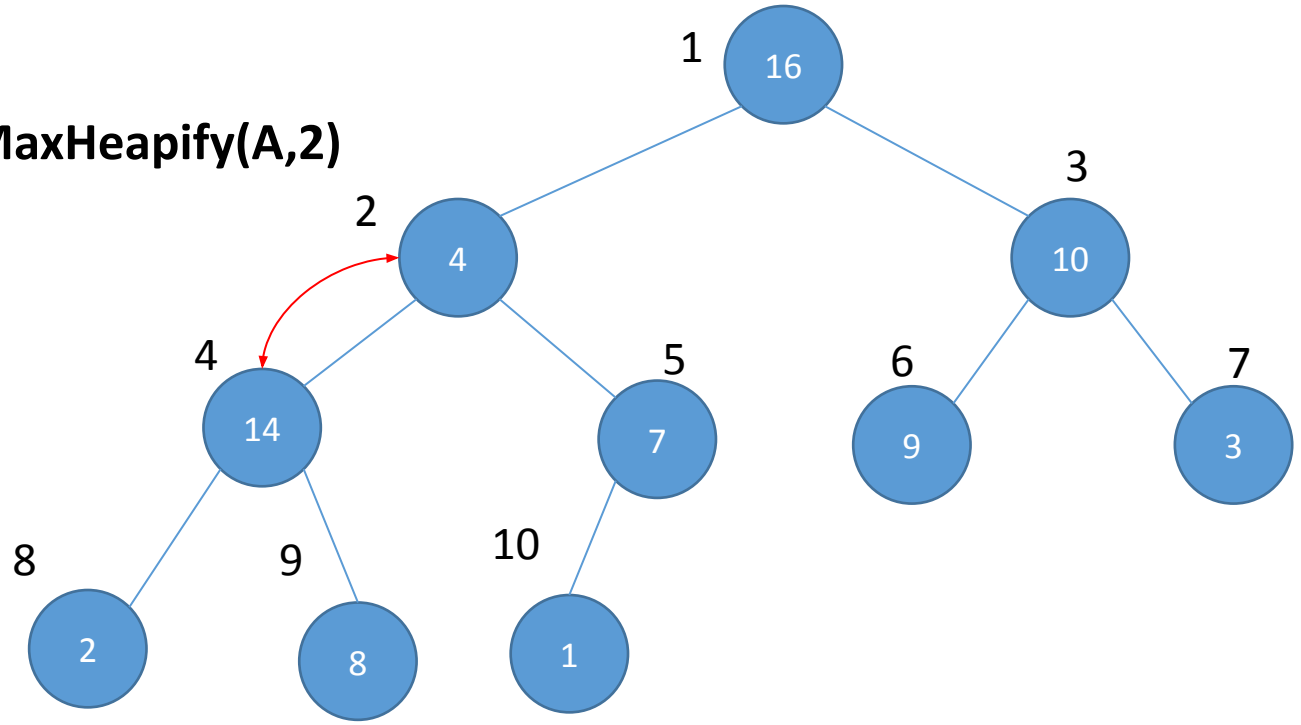
1. It is applied on an index.
2. It checks if the parent is greater than the child
3. If the parent is greater, return.
4. If the child is greater, then swap values with child
   a) Run the function on the child (recursively)

MAX HEAPIFY

- **Tries** to make the array into a max heap
- (parent value>= child value)
- MaxHeapify(A,index)

1	2	3	4	5	6	7	8	9	10
16	4	10	14	7	9	3	2	8	1

MaxHeapify(A,2)



How the function works:

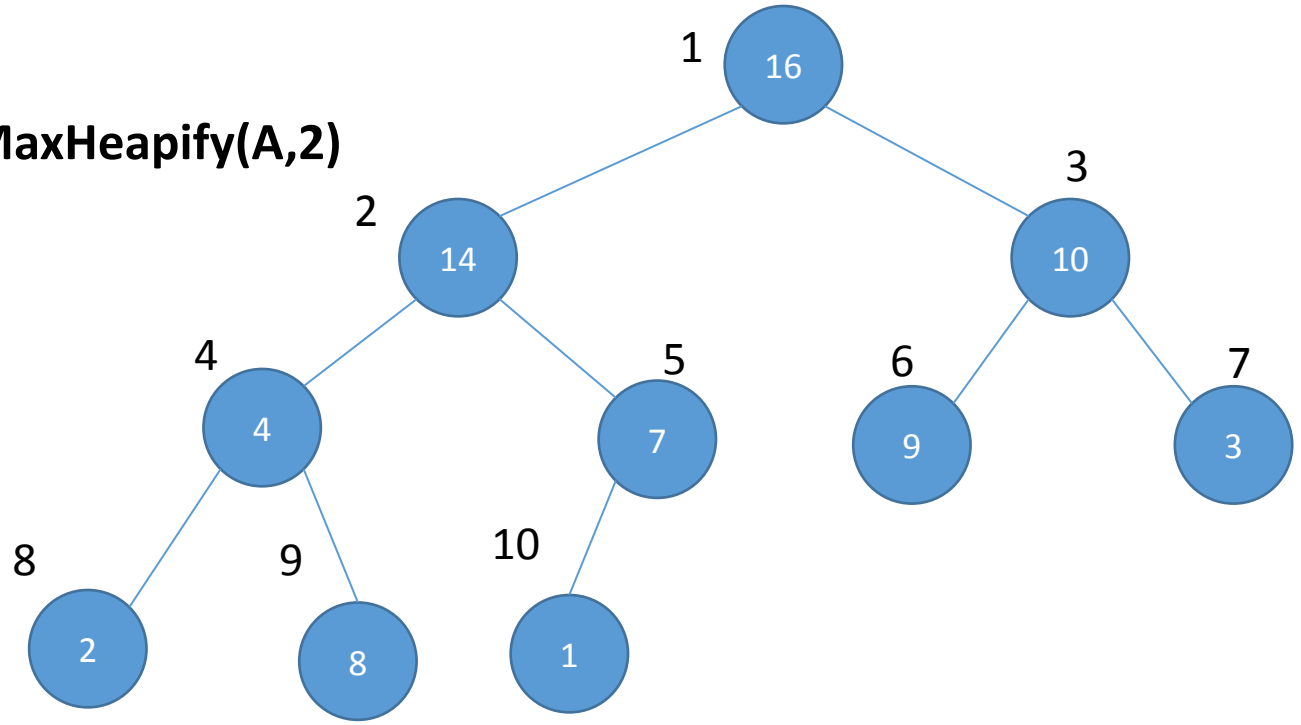
1. It is applied on an index.
2. It checks if the parent is greater than the child
3. If the parent is greater, return.
4. If the child is greater, then swap values with child
   a) Run the function on the child (recursively)

MAX HEAPIFY

- **Tries** to make the array into a max heap
- (parent value>= child value)
- MaxHeapify(A,index)

1	2	3	4	5	6	7	8	9	10
16	**14**	10	**4**	7	9	3	2	8	1

MaxHeapify(A,2)



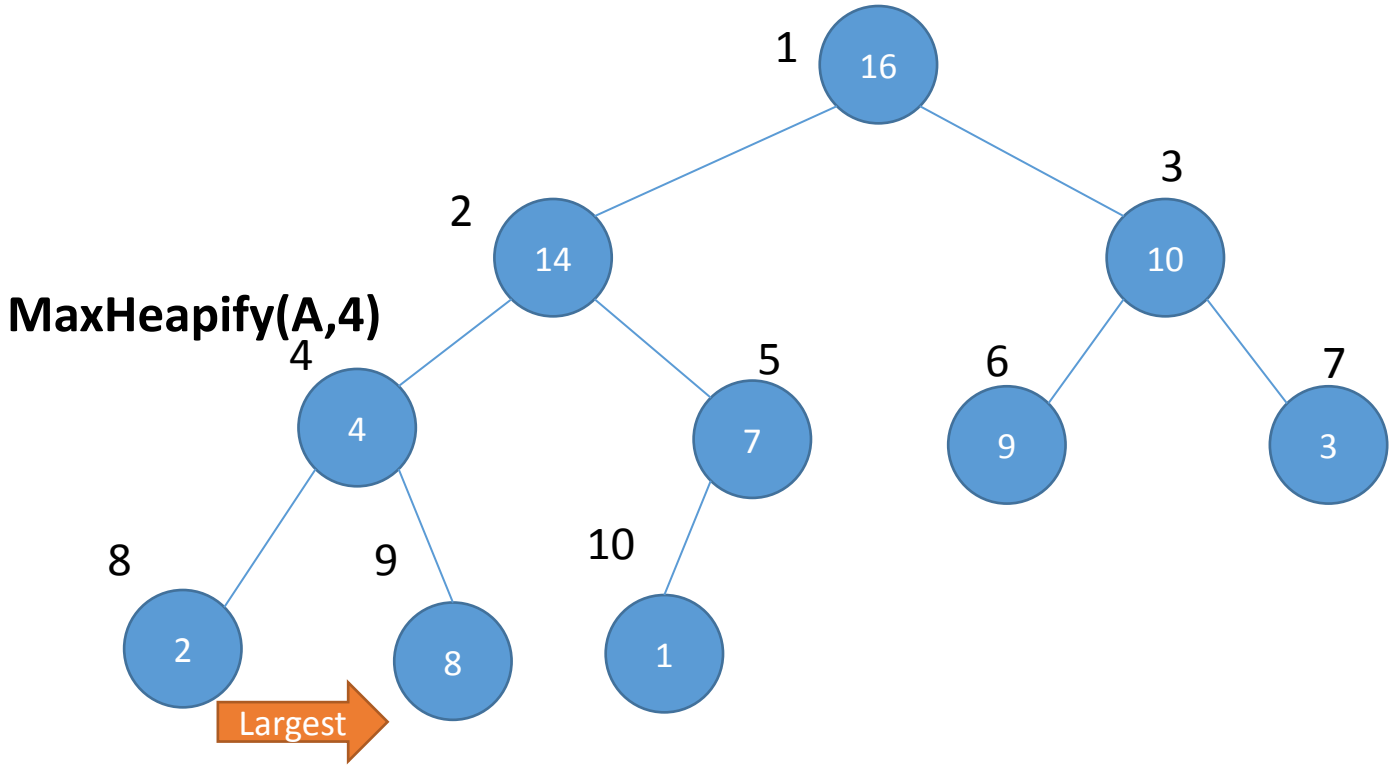
How the function works:

1. It is applied on an index.
2. It checks if the parent is greater than the child
3. If the parent is greater, return.
4. If the child is greater, then swap values with child
   a) Run the function on the child (recursively)

MAX HEAPIFY

- **Tries** to make the array into a max heap
- (parent value>= child value)
- MaxHeapify(A,index)

1	2	3	4	5	6	7	8	9	10
16	**14**	10	**4**	7	9	3	2	8	1



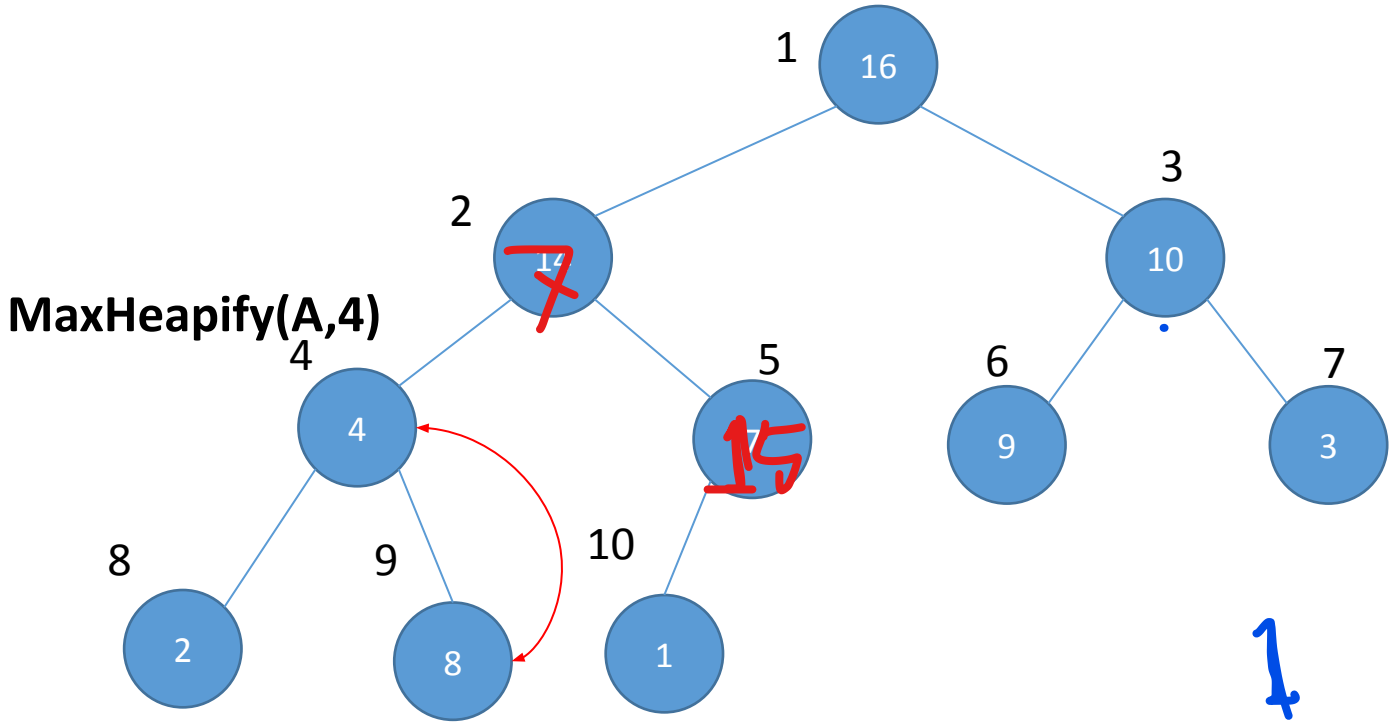
How the function works:

1. It is applied on an index.
2. It checks if the parent is greater than the child
3. If the parent is greater, return.
4. If the child is greater, then swap values with child
   a) Run the function on the child (recursively)

MAX HEAPIFY

- **Tries** to make the array into a max heap
- (parent value>= child value)
- MaxHeapify(A,index)

1	2	3	4	5	6	7	8	9	10
16	**14**	10	**4**	7	9	3	2	8	1

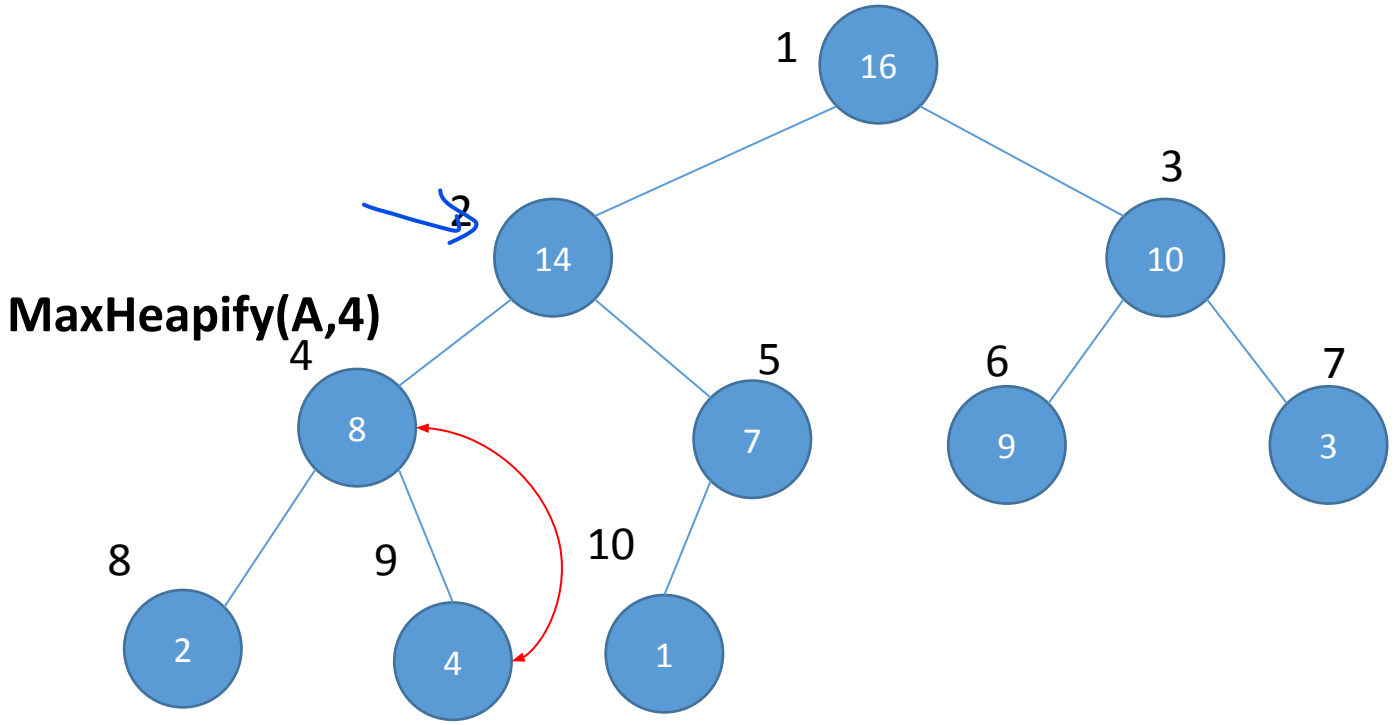


How the function works:

1. It is applied on an index.
2. It checks if the parent is greater than the child
3. If the parent is greater, return.
4. If the child is greater, then swap values with child
   a) Run the function on the child (recursively)

MAX HEAPIFY

- **Tries** to make the array into a max heap
- (parent value>= child value)
- MaxHeapify(A,index)



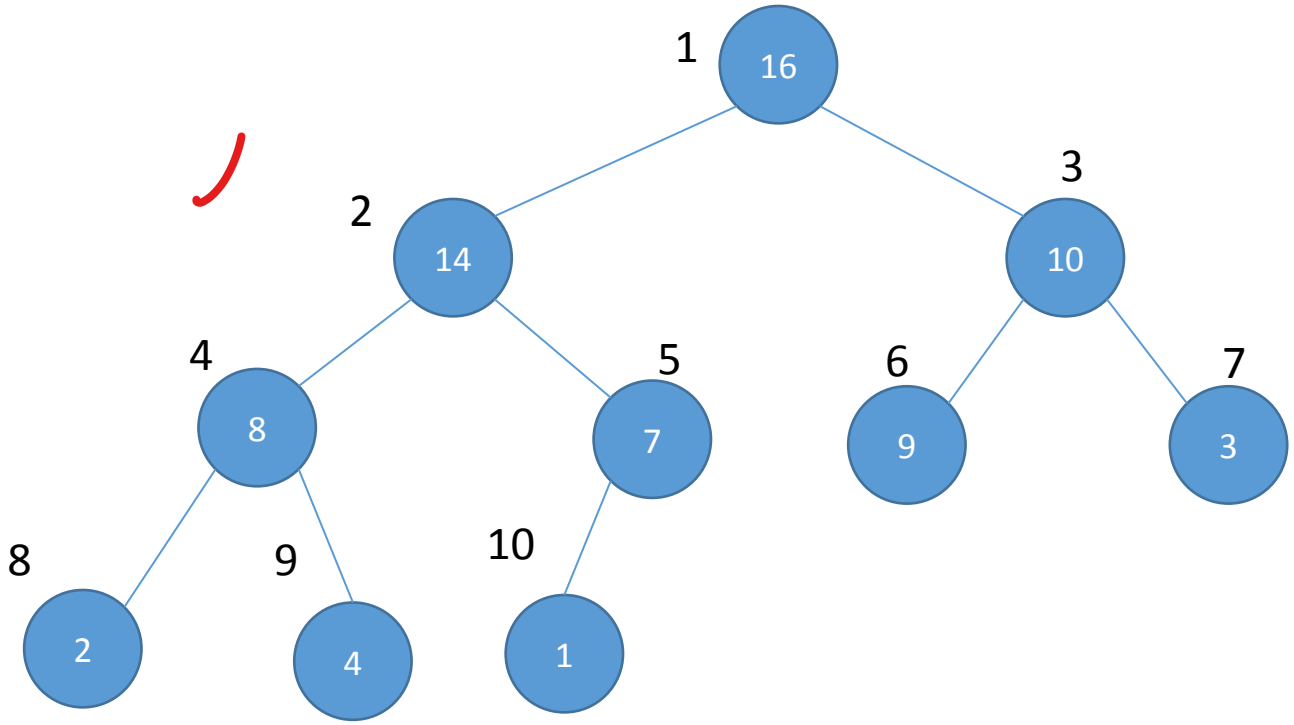
1	2	3	4	5	6	7	8	9	10
16	**14**	10	**8**	7	9	3	2	**4**	1

How the function works:

1. It is applied on an index.
2. It checks if the parent is greater than the child
3. If the parent is greater, return.
4. If the child is greater, then swap values with child
   a) Run the function on the child (recursively)

MAX HEAPIFY

- **Tries** to make the array into a max heap
- (parent value>= child value)
- MaxHeapify(A,index)



MaxHeapify(A,9)

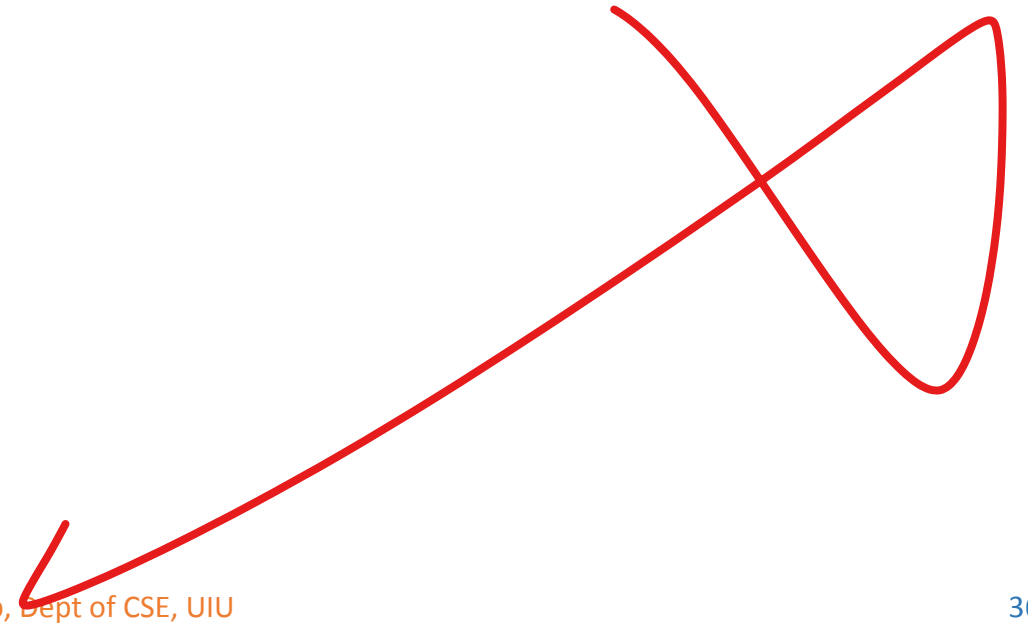
Parents

1	2	3	4	5	6	7	8	9	10
16	**14**	10	**8**	7	9	3	2	**4**	1

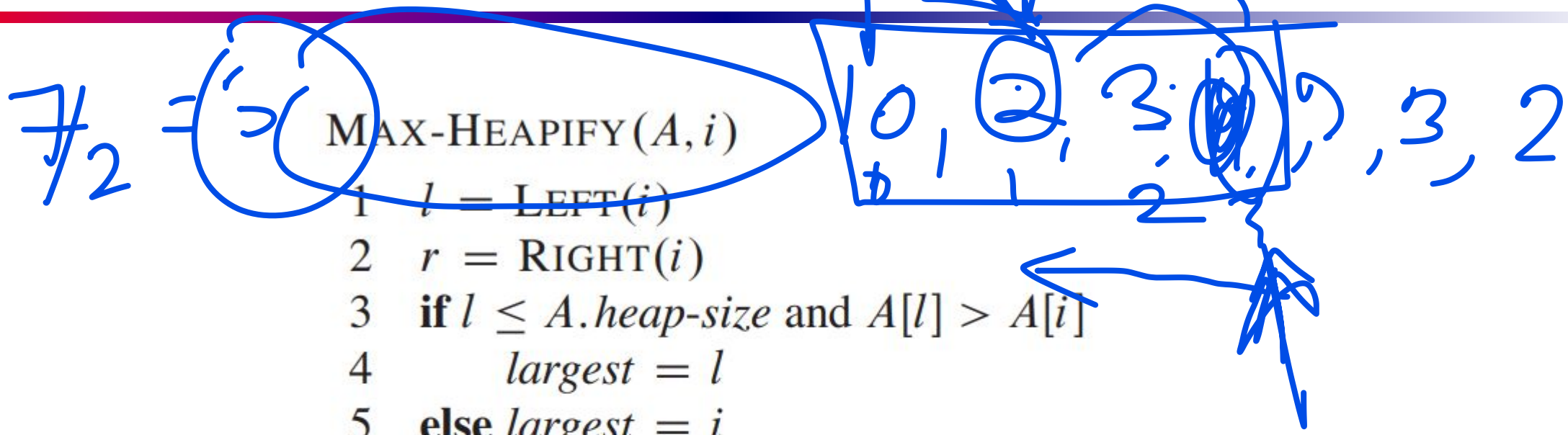
How the function works:

1. It is applied on an index.
2. It checks if the parent is greater than the child
3. If the parent is greater, return.
4. If the child is greater, then swap values with child
   a) Run the function on the child (recursively)

Let us write the function for Max Heapify



Let us write the function for Max Heapify

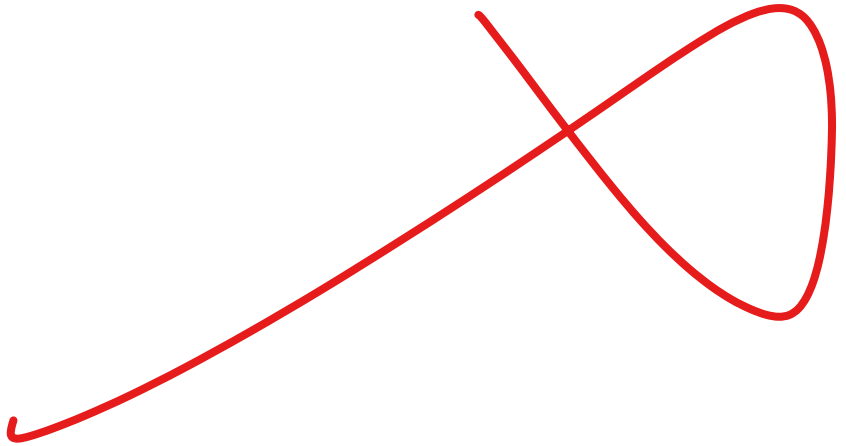


```
MAX-HEAPIFY(A, i)
1  l = LEFT(i)
2  r = RIGHT(i)
3  if l ≤ A.heap-size and A[l] > A[i]
4       largest = l
5  else largest = i
6  if r ≤ A.heap-size and A[r] > A[largest]
7       largest = r
8  if largest ≠ i
9       exchange A[i] with A[largest]
10      MAX-HEAPIFY(A, largest)
```

See git Code

Max Heapify

- It is TRYING to make the array into a heap
- But it may not fully succeed
- Let us consider the following example



Build Max Heap

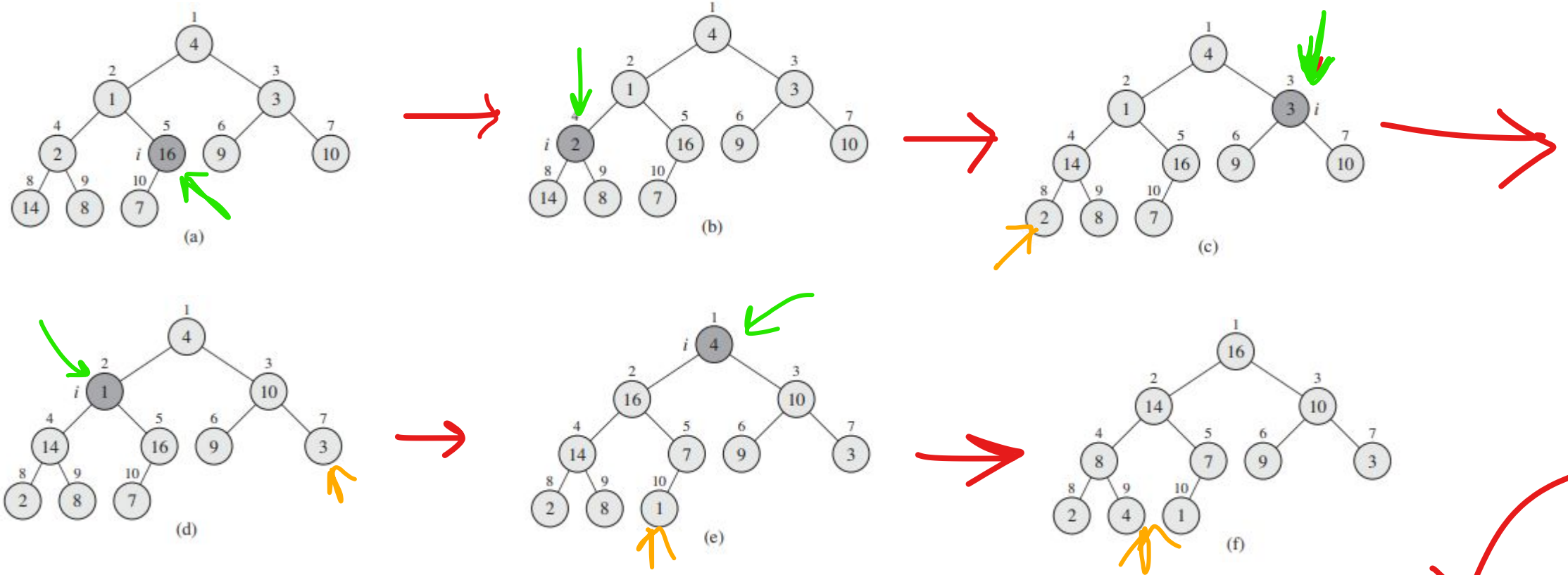
- Converts an array into a heap

```
BUILD-MAX-HEAP(A)
1  A.heap-size = A.length
2  for i = ⌊A.length/2⌋ downto 1
3      MAX-HEAPIFY(A, i)
```

Build Max Heap

- Consider the following array

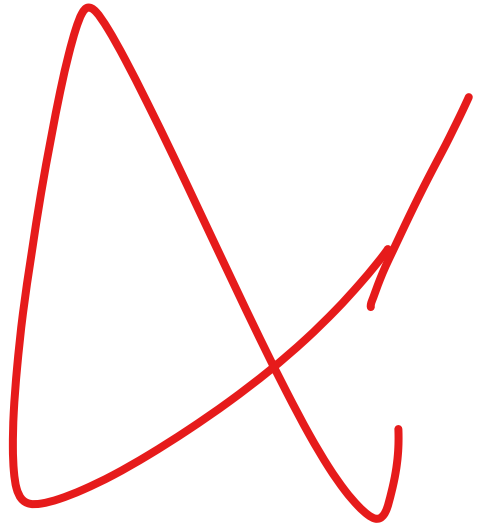
A	4	1	3	2	16	9	10	14	8	7



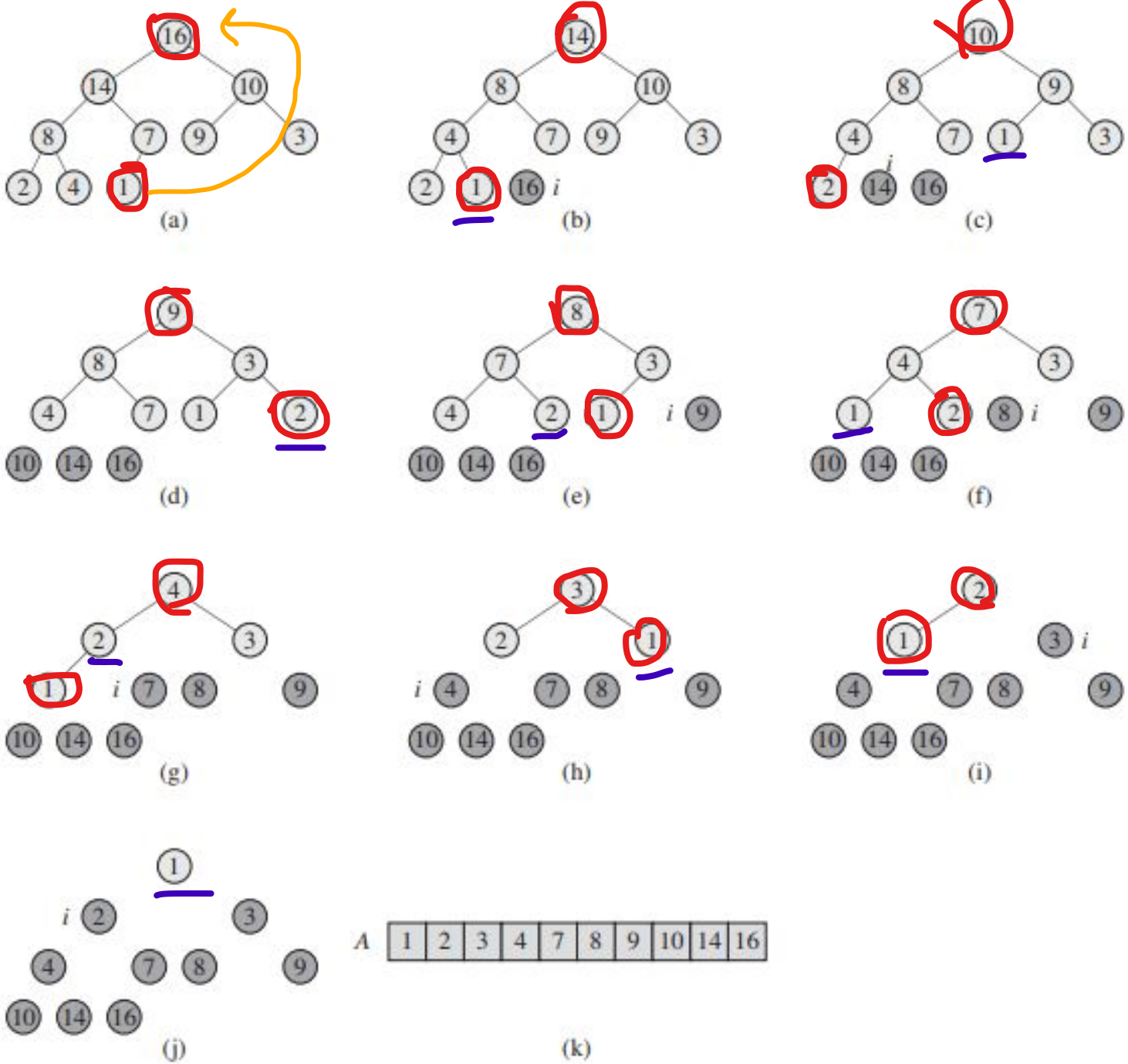
Heap Sort

```
HEAPSORT(A)
1  BUILD-MAX-HEAP(A)
2  for i = A.length downto 2
3      exchange A[1] with A[i]
4      A.heap-size = A.heap-size − 1
5      MAX-HEAPIFY(A, 1)
```

Let us Run Heap Sort At First

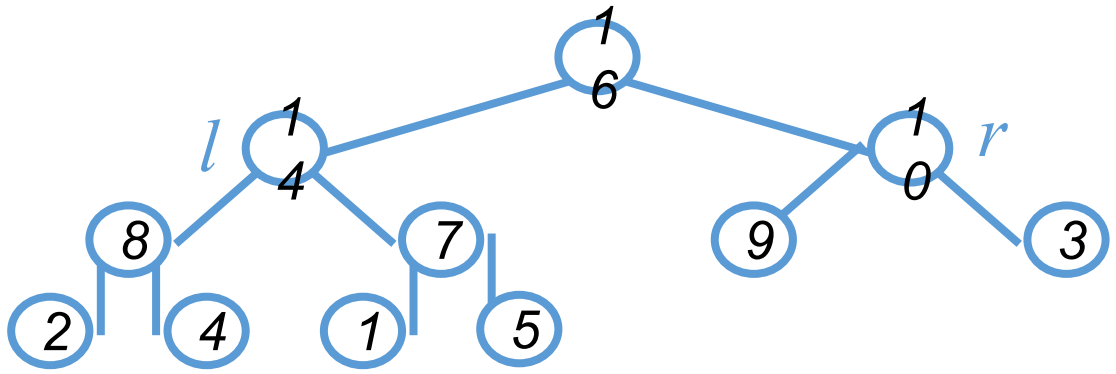


Heap Sort



Time Complexity Analysis-Max Heapify

- **Max Heapify**
  - Fixing up relationships among the elements $A[i]$, $A[l]$, and $A[r]$ takes $\Theta(1)$ time
  - *If the heap at i has n elements, at most how many elements can the subtrees at l or r have?*



- Answer: $2n/3$ (worst case: bottom row half full)
- So time taken by **`Heapify()`** is given by

  $T(n) \leq T(2n/3) + \Theta(1)$

```
MAX-HEAPIFY(A, i)
1  l = LEFT(i)
2  r = RIGHT(i)
3  if l ≤ A.heap-size and A[l] > A[i]
4       largest = l
5  else largest = i
6  if r ≤ A.heap-size and A[r] > A[largest]
7       largest = r
8  if largest ≠ i
9       exchange A[i] with A[largest]
10      MAX-HEAPIFY(A, largest)
```

Time Complexity Analysis-Max Heapify

- So we have

  $T(n) \leq T(2n/3) + \Theta(1)$

- Solving the recurrence, we have

  $T(n) = O(\lg n)$

- Thus, **`Heapify()`** takes $O(h)$ time for a node at height h

Time Complexity Analysis-Build Max Heap

- We can compute a simple upper bound on the running time of BUILD-MAXHEAP as follows.
- Each call to MAX-HEAPIFY costs O(lg n) time, and BUILDMAX-HEAP makes **O(n) such calls.**
- Thus, the running time is **O(n lg n).**

```
BUILD-MAX-HEAP(A)
1  A.heap-size = A.length
2  for i = ⌊A.length/2⌋ downto 1
3      MAX-HEAPIFY(A, i)
```

Time Complexity Analysis-Heap Sort

```
HEAPSORT(A)
1  BUILD-MAX-HEAP(A)
2  for i = A.length downto 2
3      exchange A[1] with A[i]
4      A.heap-size = A.heap-size − 1
5      MAX-HEAPIFY(A, 1)
```

- The HEAPSORT procedure takes time O(n lg n)
- since the call to BUILD-MAXHEAP takes time O(n)
- Each of the n - 1 calls to MAX-HEAPIFY takes time O(lg n)

Thank You